Software Developer Training

Full-Stack Web Development · Version 3.0

Neumann · www.neumann.ai

Full-Stack Web Development · Time allotted: **2 months**

> **What changed from V2.5 (2020):** the project is the same *idea* — an Employee Directory web app — but the stack is modernized to what teams actually ship in 2026: **React + TypeScript** on the front end, a **FastAPI (Python)** REST API on the back end, **PostgreSQL or MySQL in Docker**, real authentication, automated tests, and **AI-assisted development** as a first-class skill. The goal is unchanged: by the end you can design, build, test, and deliver a working full-stack application on your own.

---

0. How to read this document

- **Section 1–2** — what you'll build and the timeline.
- **Section 3** — the tools to install.
- **Section 4** — functional requirements (what the app must do).
- **Section 5** — technical requirements (how to build it).
- **Section 6** — using AI to help you learn and build (new in V3).
- **Section 7** — deliverables, documentation, and testing.
- **Section 8** — Month 2: stretch goals and "make it production-grade".
- **Section 9** — grading rubric.
- **Appendix A** — seed data. **Appendix B** — curated learning links.

A **working application is mandatory.** Polish (animation, responsive design, accessibility, UX) is a strong plus. Deliver a Git repository (not a zip) with a clear README, a short design document, and a test-cases document.

---

1. The project — "Neumann Directory"

You will build a **company employee/contact directory** — a single-page web application backed by a REST API and a database. A user logs in, browses a responsive grid of employee cards, filters and searches the list, opens a details panel, and can **create, edit, and delete** employees. Each employee carries a brand color that themes their card and detail panel.

This is a classic **CRUD** application — the bread-and-butter of business software. It is deliberately small in scope but touches every layer you need to understand: UI components and state, HTTP, validation, authentication, a relational schema, migrations, and tests.

The three core screens (see `docs/figures/` for reference mockups)

1. **Login** — email + password, then enter the app.
2. **Directory (main)** — fixed header + collapsible left filter sidebar; a responsive, scrolling grid of employee cards. Cards-per-row scales with screen width. The card's bottom bar uses the employee's brand color.

3. **Details / Add / Edit** — a panel that slides in from the right with the full record. On desktop it pushes the grid aside; on mobile it overlays. The same panel, in "form mode," handles **Add** and **Edit**. **Delete** asks for confirmation first.

---

2. Timeline (suggested, 2 months)

Month 1 — Build the core app

Phase	Days	What you do
1. Research & setup	4	Stand up the dev environment (Docker, Node, Python). Work through short tutorials on React, TypeScript, FastAPI, SQL. Run the "hello world" of each. Use AI to explain concepts you don't understand (§6).
2. Design document	1.5	Write the design doc: component tree, API contract (endpoints + request/response shapes), database schema, and how the pieces talk. Review with your team lead before coding.
3. Implementation	~17	Build backend → frontend → wire them together. Follow the progressive path in §5.1. Write unit tests as you go, not at the end.
4. Test-cases document	1.5	Write out test cases (scenario, steps, expected vs. actual, pass/fail) for each module.
5. Validation & fixes	rest	Demo to your team lead, fix what's flagged, polish.

Month 2 — Make it real

Pick from §8 (auth hardening, pagination, server-side search/sort/filter, image upload, CI, the AI feature, deployment). Treat Month 2 as "take the prototype to something you'd be comfortable shipping."

---

3. Required software

Tool	Why	Notes
Docker Desktop	Runs Postgres/MySQL (and optionally the whole app) in containers — no "works on my machine."	This is the single biggest modernization vs. V2.5. Learn `docker compose up`.
Node.js 20 LTS + npm	Runs the React/Vite frontend.	Use the LTS. `nvm` (or `nvm-windows`) is handy.
Python 3.12	Runs the FastAPI backend.	Use a virtual environment (`venv`). 3.11+ required.
VS Code	Editor.	Extensions: Python, Pylance, ESLint, Prettier, Docker, "Even Better TOML".
Git + a GitHub account	Version control — **mandatory**, commit often.	The deliverable is a repo with real history, not a zip.
A modern browser	Chrome / Edge / Firefox.	You'll live in the DevTools (Network, Console, React DevTools).
An API client	Insomnia, Postman, or the built-in **Swagger UI** FastAPI generates.	You'll test the API before the UI exists.
(Optional) Anthropic / Claude API key	For the optional AI feature in §6.3 and §8.	Free-tier-friendly; the app runs fine without it.

You do **not** need Photoshop. Mockups are provided as images; if you want to design, **Figma** (free) is the modern choice.

---

4. Functional requirements

4.1 Authentication

- **Login** screen (email + password). On success the app stores a token and shows the directory; on failure it shows a clear error.
- **Logout** clears the session.
- Protected screens redirect to login when there is no valid session.
- (Month 2) **Register** a new account, and role-based access: an **admin** can edit/delete; a **viewer** is read-only.

4.2 Directory (list)

- Display employees as a **responsive grid of cards**. Number of cards per row adapts to viewport width.
- Each card shows: photo (or a generated avatar/initials fallback), name, company, city, and a **bottom bar tinted with the employee's brand color**.
- **Loading state**: show a skeleton/spinner while data loads; fade the content in when ready. (No animated-GIF loader required — use CSS skeletons.)
- **Empty state**: a friendly message when there are no results.

- **Search**: a search box filters by name / company / city. (Month 1: client-side is fine. Month 2: server-side.)
- **Company filter**: a checkbox list of companies in the left sidebar filters the grid. Selecting/deselecting animates items in/out.
- The left sidebar is **fixed** while the grid scrolls. On mobile, the header collapses to a menu (only the logo shows) and the sidebar is hidden behind a toggle.

4.3 Details panel

- Clicking a card **slides a details panel in from the right** with the full record. The selected card is highlighted (e.g., a colored border).
- The panel's accent (bar/border) uses the employee's brand color.
- Closing slides it back out. On desktop the panel pushes the grid left; on mobile it overlays the grid.

4.4 Create / Edit / Delete

- **Add**: a button opens the panel in form mode with empty fields. Validate on the client *and* the server. On save, the new employee appears in the grid (no full page reload).
- **Edit**: opens the same form pre-filled. The employee **ID is read-only**. Save updates the card in place.
- **Delete**: a delete button with a **confirmation dialog** ("Are you sure?"). Deleting removes the card.
- All four operations go through the REST API and reflect in the database.

4.5 Form validation rules (minimum)

- Required: first name, last name, company.
- Email (if present) must be a valid email.
- Brand color must be a valid hex color (e.g., `#8bc447`).
- Show inline, field-level error messages.

---

5. Technical requirements

5.1 Build it progressively (recommended order)

1. **Backend with seed data** → get `GET /employees` returning JSON from the database (Swagger UI proves it).
2. **Frontend reads the API** → render the grid from real data.
3. **Details + search + filter** → read-only interactions.
4. **Auth** → login gate.
5. **Create / Edit / Delete** → full CRUD.
6. **Tests, polish, responsive, accessibility.**

Don't try to build everything at once. Get one thin vertical slice working end-to-end, then widen it.

5.2 Frontend

- **React 18 + TypeScript**, built with **Vite**. Functional components and **hooks** only (no class components).
- **Routing**: React Router (`/login` , `/` , `/employees/:id`).
- **Data fetching**: use **TanStack Query (React Query)** for server state (caching, loading/error states, refetch after mutations). This replaces hand-rolled `fetch` + `useState` spaghetti.
- **HTTP**: `fetch` or `axios` wrapped in a small typed API client.
- **Styling**: modern CSS — CSS Modules or plain CSS with custom properties (CSS variables) for theming. Tailwind is acceptable if you prefer it. Animations via CSS transitions; the slide-in panel via CSS `transform` , not a jQuery plugin.
- **Type safety**: model the API responses as TypeScript types/interfaces. No `any` in component props.
- **Responsiveness**: CSS Grid / Flexbox + media queries (or container queries). Must work from ~360px (mobile) to desktop.
- **Accessibility (plus, strongly encouraged)**: semantic HTML, labels tied to inputs, keyboard-navigable, focus management on the panel, `aria-*` where needed.
- **Font**: Mulish (kept from V2.5) or another Google Font of your choice.

> **Explicitly retired from V2.5:** pure HTML/CSS/JS, jQuery, Bootstrap, jQuery AJAX, animated-GIF loaders. These were 2015-era choices; the modern equivalent is React + a component model + CSS3 + `fetch` .

5.3 Backend

- **FastAPI** (Python) — async, typed, and it **auto-generates OpenAPI/Swagger docs** at `/docs` .
- **Pydantic v2** schemas for request/response validation (this is your server-side validation).
- **SQLAlchemy 2.0** ORM for the data layer, with **Alembic** for database migrations.
- **Auth**: JWT bearer tokens. Passwords hashed with **bcrypt** (via `passlib`) — never stored in plaintext. (`python-jose` or `pyjwt` for tokens.)
- **Config** via environment variables (`.env` , read with `pydantic-settings`). No secrets in code.
- **CORS** configured so the Vite dev server can call the API.
- All responses are **JSON**. All communication is over HTTP(S). Errors return proper status codes (400/401/403/404/422) with a JSON body.

Required endpoints (the API contract)

Method	Path	Purpose
`POST`	`/auth/register`	Create an account (Month 2 / optional Month 1).
`POST`	`/auth/login`	Exchange email+password for a JWT.
`GET`	`/auth/me`	Current user from the token.
`GET`	`/employees`	List employees. Supports `?search=`, `?company=`, `?page=`, `?page_size=`, `?sort=` (server-side in Month 2).
`GET`	`/employees/{id}`	One employee.
`POST`	`/employees`	Create (auth required).
`PUT`	`/employees/{id}`	Update (auth required).
`DELETE`	`/employees/{id}`	Delete (auth required, admin).
`GET`	`/companies`	Distinct company list (for the filter).
`GET`	`/health`	Liveness check.

5.4 Database — PostgreSQL (recommended) or MySQL, both via Docker

You asked which is better. Here's the honest answer and **both options are provided** in the reference solution.

	PostgreSQL (recommended)	MySQL
Standards & data types	Richer (native JSON/JSONB, arrays, `uuid`, strict types).	Good, slightly less strict historically.
Learning value	Industry default for new backend projects; what you'll most likely meet at work.	Still everywhere (WordPress, legacy, LAMP shops).
Tooling with SQLAlchemy/Alembic	First-class.	First-class.
Why it's in this training	**Default.** Teaches you the modern default and pairs naturally with Docker.	Provided as an alternative so you can see the code is DB-agnostic.

Recommendation: use PostgreSQL in Docker. Because we use SQLAlchemy + Alembic, **switching is one line in** `.env` (`DATABASE_URL`) — the reference repo ships a `docker-compose.yml` (Postgres) and a `docker-compose.mysql.yml` (MySQL) so you can run either and prove the app doesn't care. Understanding *why* it doesn't care (the ORM abstracts the dialect) is itself a learning objective.

The database must be seeded with the default data in **Appendix A** on first run (a `seed` script / migration).

5.5 Project layout (reference)

```
neumann-directory/
├─ docker-compose.yml            # Postgres + adminer + (optionally) api & web
├─ docker-compose.mysql.yml      # MySQL variant
├─ backend/                      # FastAPI app, SQLAlchemy, Alembic, pytest
├─ frontend/                     # React + Vite + TS, Vitest/RTL
├─ docs/                         # design doc, test cases, figures
└─ README.md
```

---

6. Using AI to help you (new in V3)

Modern developers build *with* AI assistants. Part of this training is learning to use them **well** — as a tutor, a pair-programmer, and a reviewer — without letting them think *for* you. **You must be able to explain every line you submit.** "The AI wrote it" is not an acceptable answer in your demo.

6.1 AI as a learning tutor (during research & whenever stuck)

- "Explain what a React hook is, with a tiny example."
- "What's the difference between PostgreSQL and MySQL for this project?"
- "I get a CORS error calling my API from the Vite dev server — what does that mean and how do I fix it?"
- Paste an error message and ask for the *cause*, not just the fix.

6.2 AI as a pair-programmer (during implementation)

- Scaffold boilerplate (a Pydantic schema, a React form component) then **read and adapt it** — don't paste blindly.
- "Write a pytest test for this `create_employee` endpoint."
- "Here's my component — suggest accessibility improvements."
- Use an AI coding tool (Claude Code, Copilot, Cursor) if available, but keep ownership of the design.

Rules of engagement (you will be graded on this): 1. **Understand before you commit.** If you can't explain it, don't ship it. 2. **Verify.** AI hallucinates APIs and library methods. Check the docs; run it. 3. **Security & secrets.** Never paste real credentials or customer data into a prompt. 4. **Attribute heavy lifting** in your design doc ("scaffolded the form with AI, then refactored X/Y").

6.3 AI *inside* the product (optional feature — great Month-2 stretch)

Add a real AI feature to the app. The reference solution includes an optional, env-gated endpoint:

- **`POST /employees/{id}/bio`** — generates a short, friendly professional bio for an employee from their structured fields, using the **Claude API** (`claude-opus-4-8`). It's disabled unless `ANTHROPIC_API_KEY` is set, so the app runs without it.

This teaches you to integrate a third-party API, handle keys/secrets, deal with latency (loading states), and degrade gracefully when a feature is off. Other ideas: natural-language search ("show me everyone in Chicago"), or auto-suggesting a brand color from the company name.

---

7. Deliverables, documentation & testing

7.1 Deliverables

1. A **Git repository** (push to GitHub) with meaningful commit history.
2. A working app that runs with documented steps (`docker compose up` , plus how to start frontend/backend).
3. A **README** with setup, run, and test instructions.
4. A **Design Document** (`docs/DESIGN.md`).
5. A **Test-Cases Document** (`docs/TEST_CASES.md`).

7.2 Design document — should briefly describe

- Architecture overview (a diagram is great): frontend ↔ API ↔ database.
- The **component tree** (which React components exist and how they nest).
- The **API contract** (endpoints, request/response shapes, status codes).
- The **database schema** (tables, columns, types, relationships).
- Key decisions and trade-offs (incl. Postgres-vs-MySQL choice, and where you used AI).
- Comment and indent your code as you go — readable code is part of the grade.

7.3 Testing — two layers

- **Automated tests (required):**
- **Backend**: `pytest` covering the CRUD endpoints, auth, and validation (use a test database / SQLite or a disposable Postgres).
- **Frontend**: **Vitest + React Testing Library** for a few key components (e.g., the card renders, the form validates). (Month 2: an end-to-end test with **Playwright** is a strong plus.)
- **Test-cases document (required):** for each module, a table of
- **Test Scenario** (what's being tested)
- **Steps** (how to perform it)
- **Expected Result** vs **Actual Result**
- **Pass / Fail**

---

8. Month 2 — stretch goals (pick several)

These mirror the original V2.5 Month-2 tasks, modernized, plus new "production-grade" goals.

1. **Company checkbox filter** *(was Task 1)* — done server-side via `?company=` .
2. **Pagination / infinite scroll** *(was Task 2)* — "imagine 10,000 users." Server-side `?page=&page_size=` . Bonus: cursor-based pagination or infinite scroll.

3. **Edit & Delete** *(was Tasks 3–4)* — if not already finished in Month 1; ID stays read-only; delete needs confirmation.
4. **Login + accounts + roles** *(was Task 5)* — register, JWT, admin vs. viewer permissions.
5. **Server-side search & sort** — push search/sort into SQL; debounce the search box.
6. **Avatar/photo upload** — file upload endpoint + storage (local or S3-compatible); default-avatar fallback.
7. **Dark mode** + a polished design system (CSS variables, tokens).
8. **CI** — a GitHub Actions workflow that runs `pytest` and `vitest` on every push.
9. **Dockerize the whole app** — `docker compose up` brings up db + api + web together.
10. **The AI feature** (§6.3).
11. **Deploy it** — to a free tier (Render/Railway/Fly.io for the API+DB, Vercel/Netlify for the frontend) and put the live URL in the README.
12. **Observability & hardening** — request logging, rate limiting on `/auth/login`, input sanitization, `.env.example`, no secrets in Git.

---

9. Grading rubric (what "good" looks like)

Area	Weight	What earns marks
Working app (all core features)	30%	Login, list, details, search, filter, full CRUD all work against the real DB.
Code quality & structure	20%	Readable, typed, sensibly organized, commented; no dead code; consistent style.
Correct use of the stack	15%	Idiomatic React + hooks + React Query; FastAPI + Pydantic + SQLAlchemy + Alembic; migrations; JWT auth done right.
Testing	15%	Meaningful automated tests pass; thorough test-cases doc.
Documentation	10%	Clear README + design doc; anyone can run it from your instructions.
UX / responsiveness / a11y	5%	Looks good, works on mobile, keyboard-usable, loading/empty/error states.
AI usage (judgment, not just output)	5%	Used AI to accelerate *and* can explain everything; followed the rules in §6.2.
Bonus	+10%	Deployed live, CI, Docker-all, AI feature, Playwright e2e, dark mode.

Figures — interface mockups

Reference mockups of the target UI. These are renders of the worked reference solution; trainees should match the **layout and behavior**, not pixel-copy the styling (styling polish is a plus, §9).

Figure 1 — Login

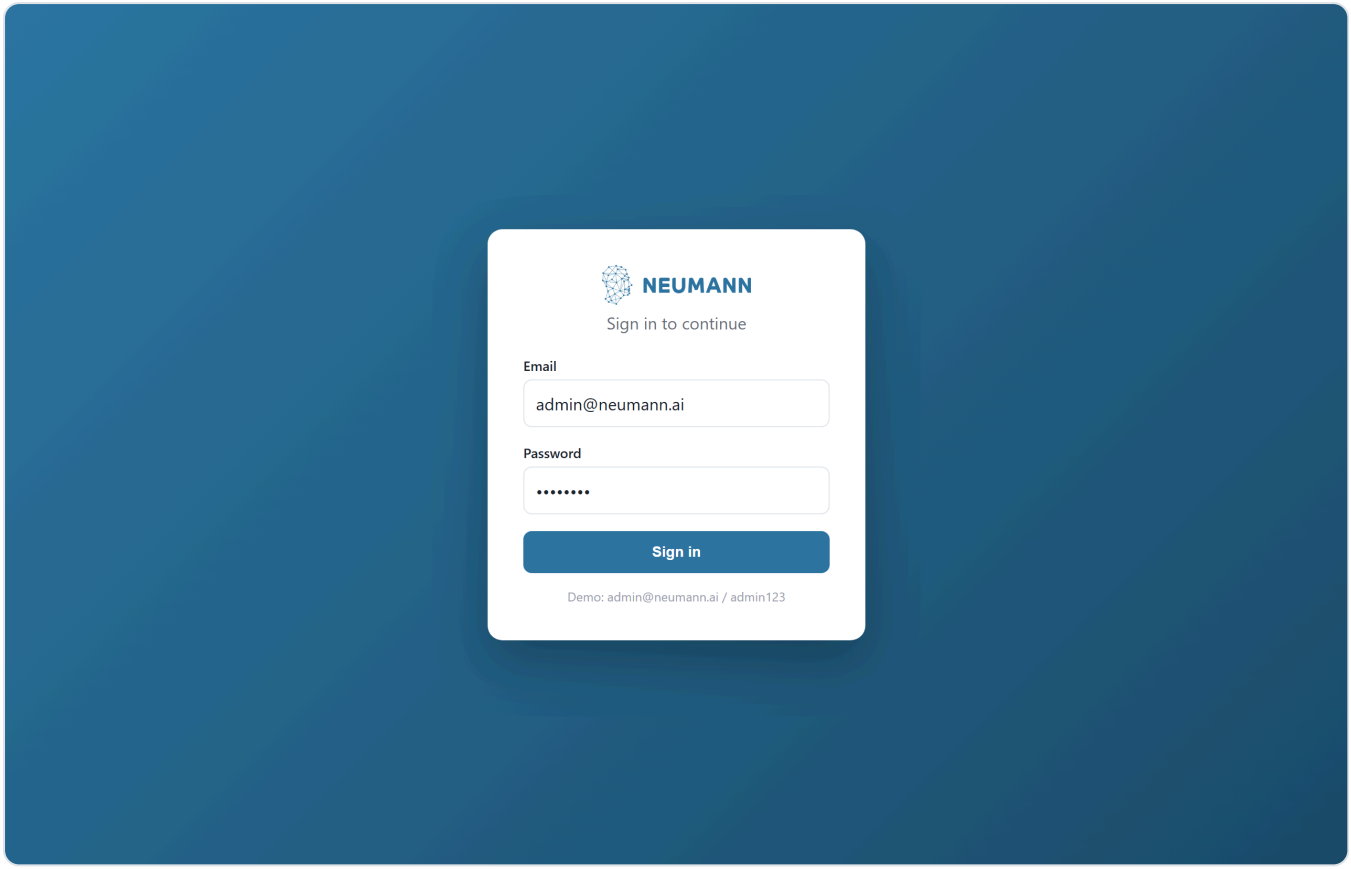


Figure 2 — Directory (desktop): fixed header with the Neumann logo, a fixed left company filter, and a responsive card grid. Each card shows initials/photo, name, title, company·city, and a brand-colored bottom bar.

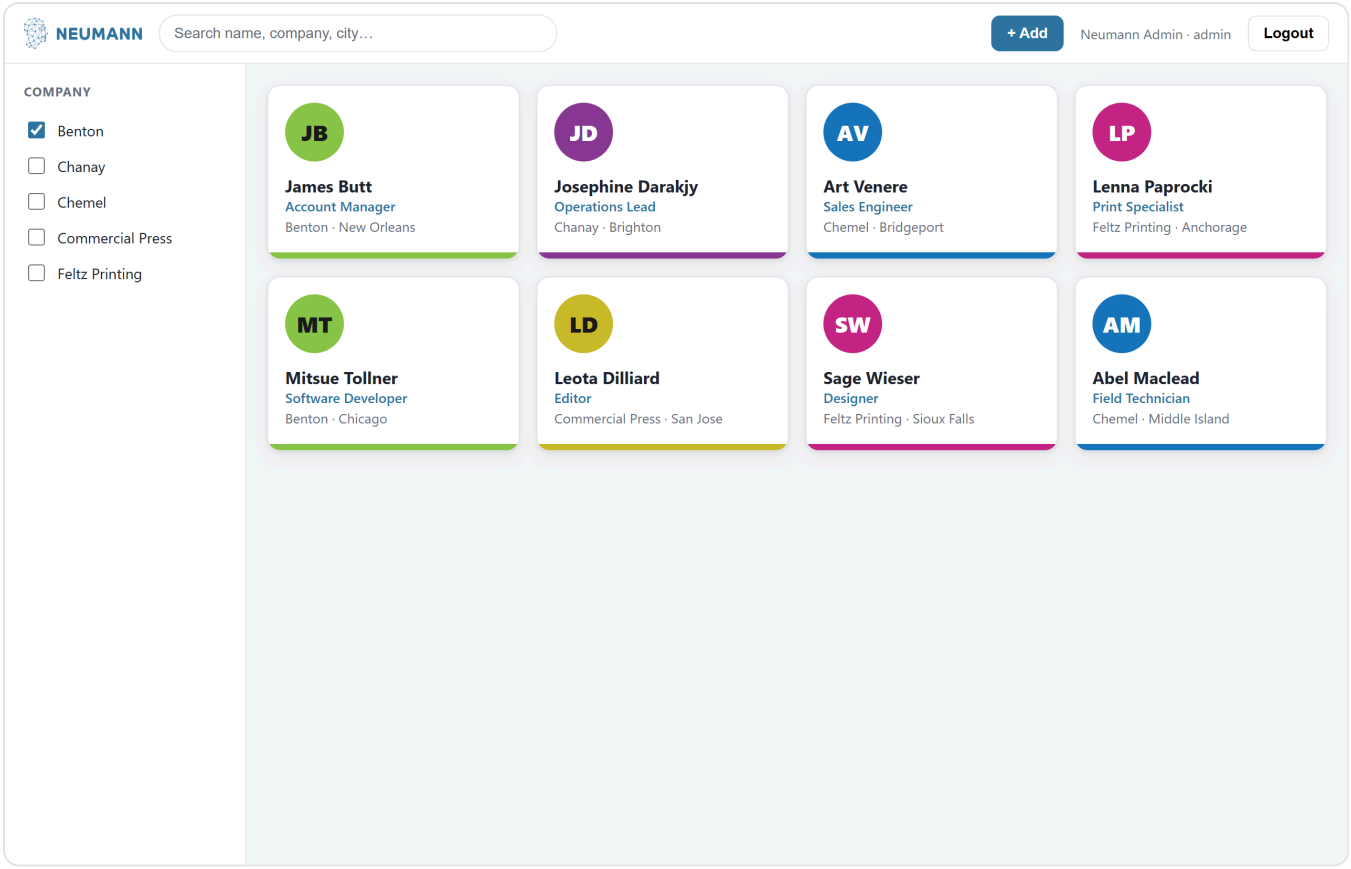


Figure 3 — Details (desktop): clicking a card highlights it and slides in a details panel from the right, themed with the employee's brand color, including the optional "Generate with AI" bio action.

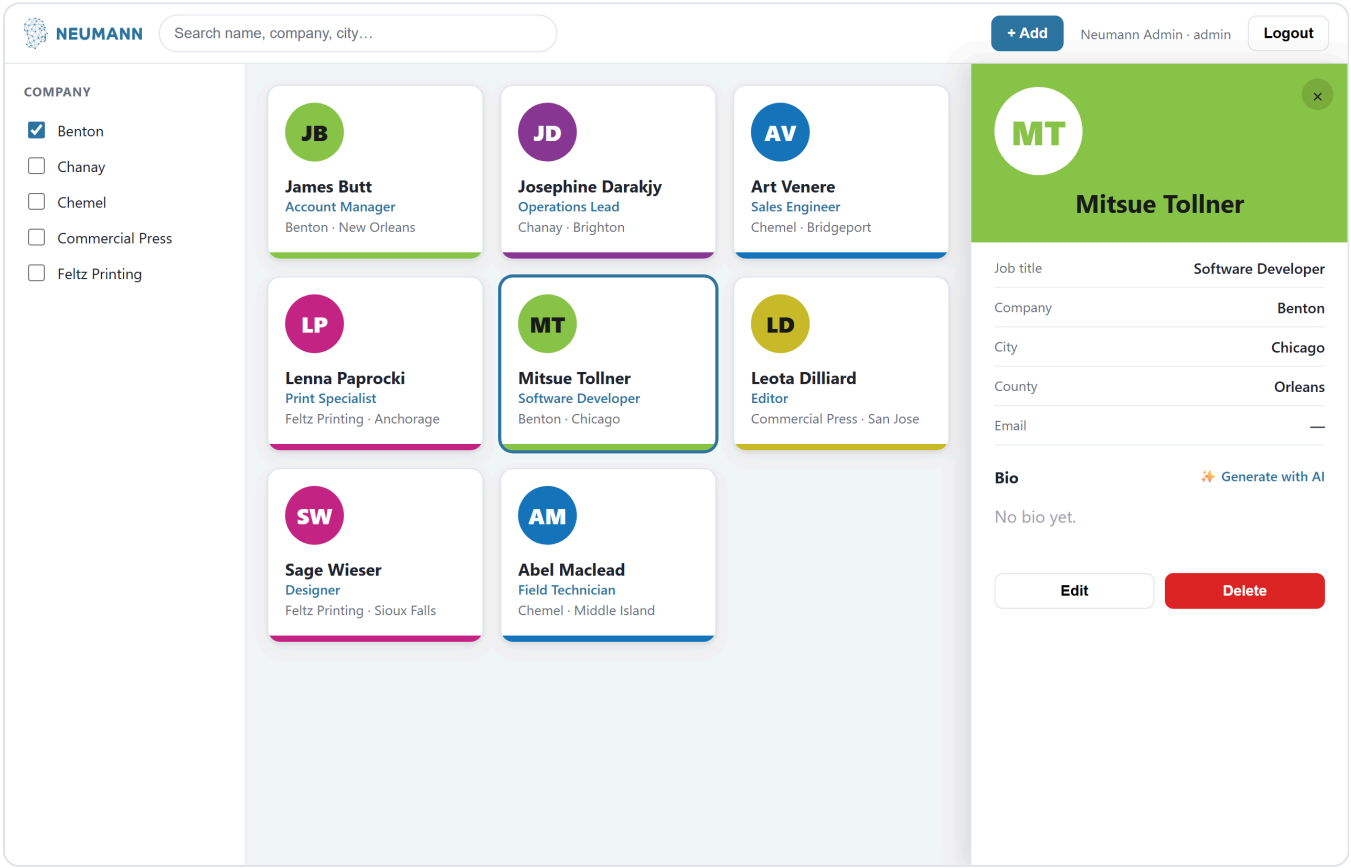


Figure 4 — Add / Edit (desktop): the same panel in form mode. On edit, fields are pre-filled and the Employee ID is read-only.

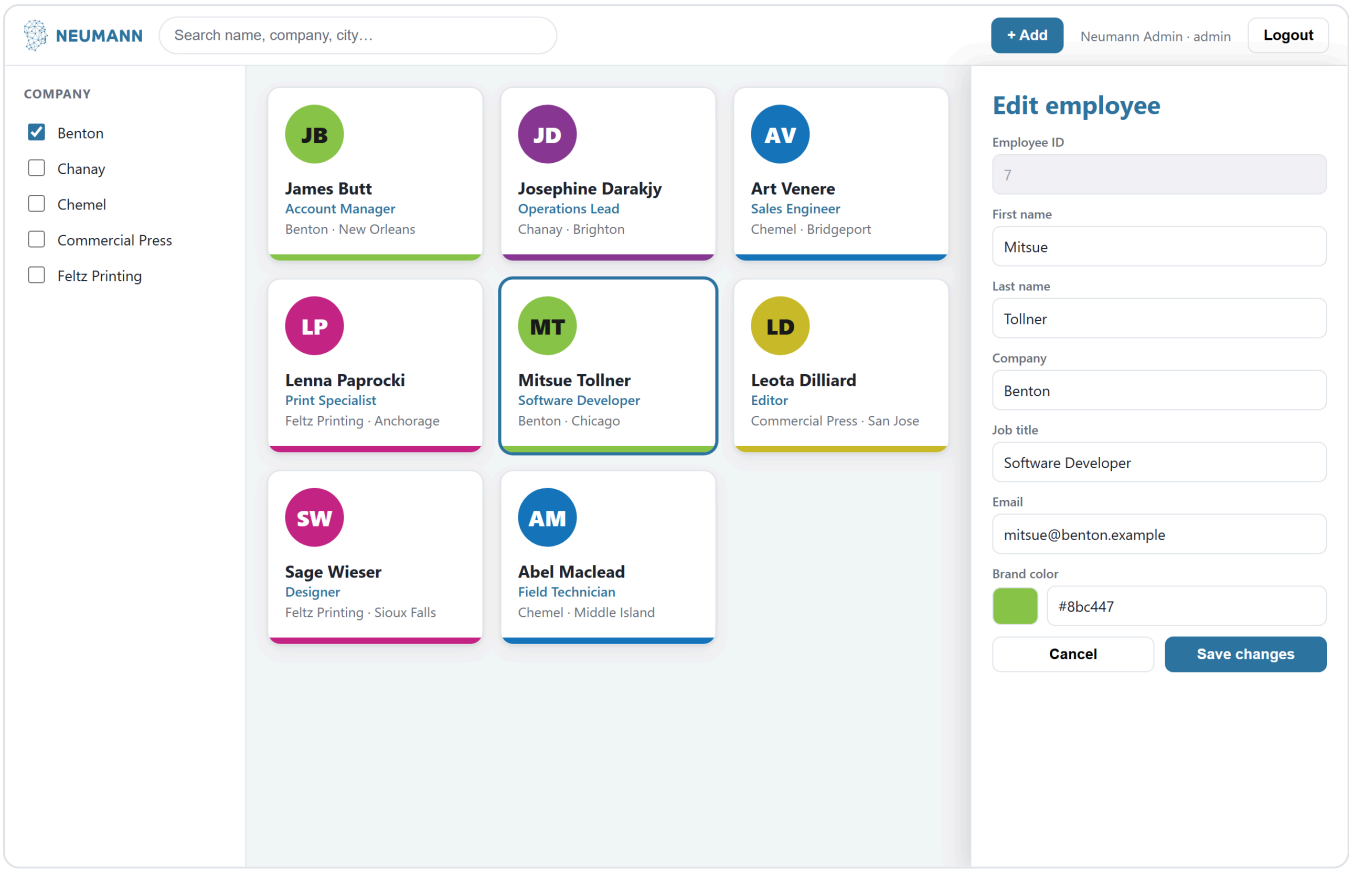


Figure 5 — Directory (mobile): the header collapses to a menu + logo; the filter sidebar hides behind the menu; cards stack to one column.

NEUMANN

+ Add

JB

James Butt

Account Manager

Benton · New Orleans

JD

Josephine Darakjy

Operations Lead

Chanay · Brighton

AV

Art Venere

Sales Engineer

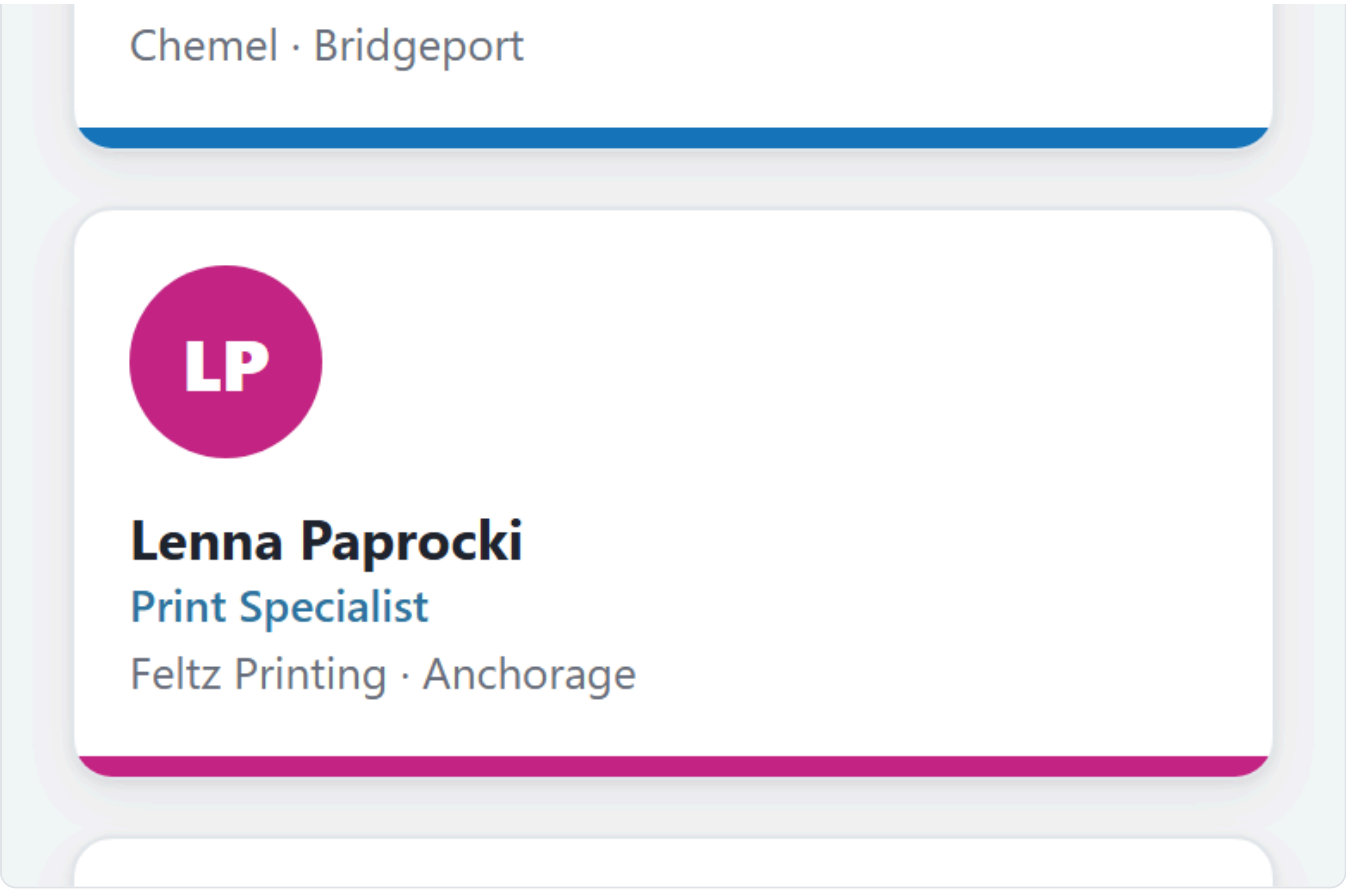
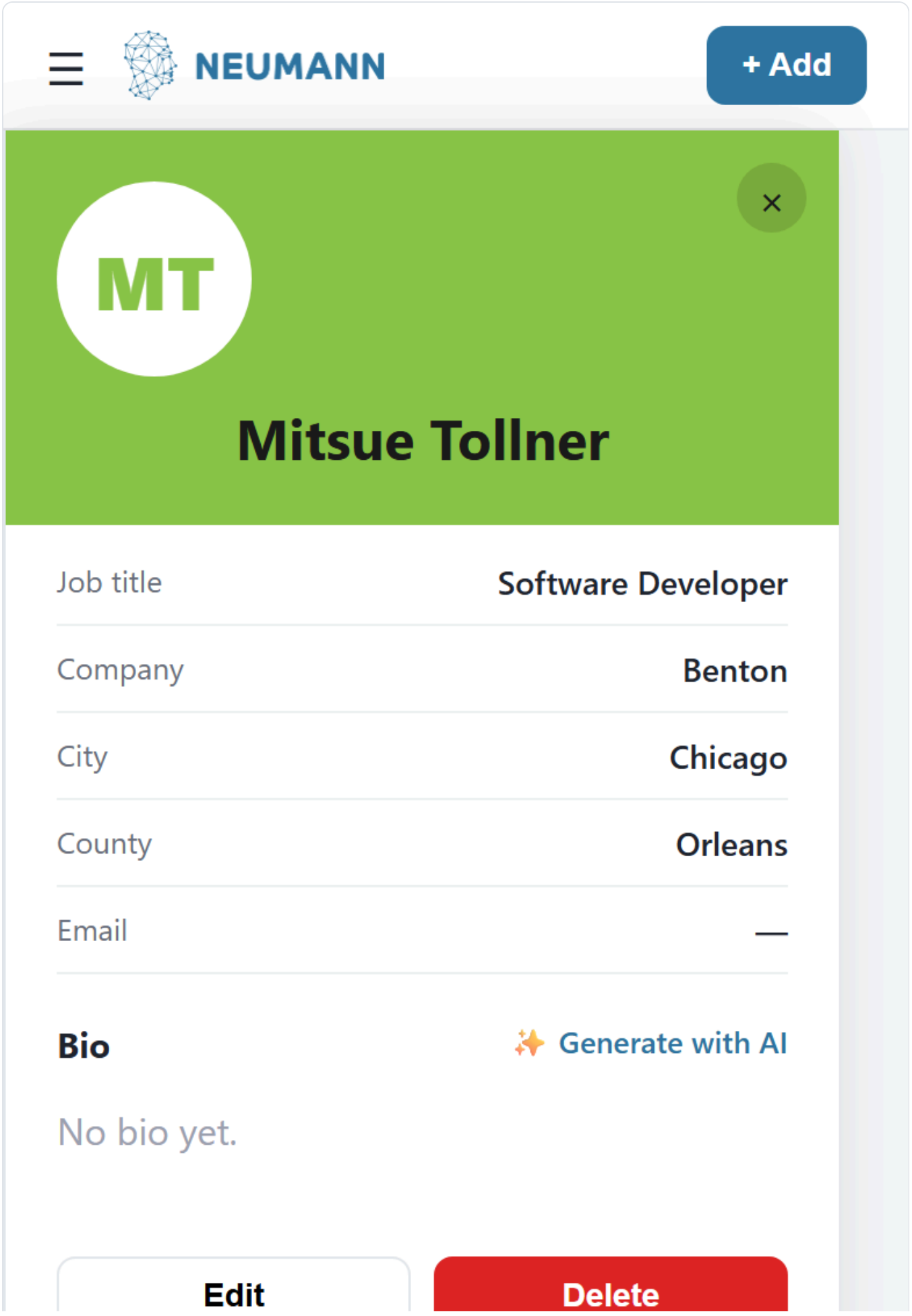
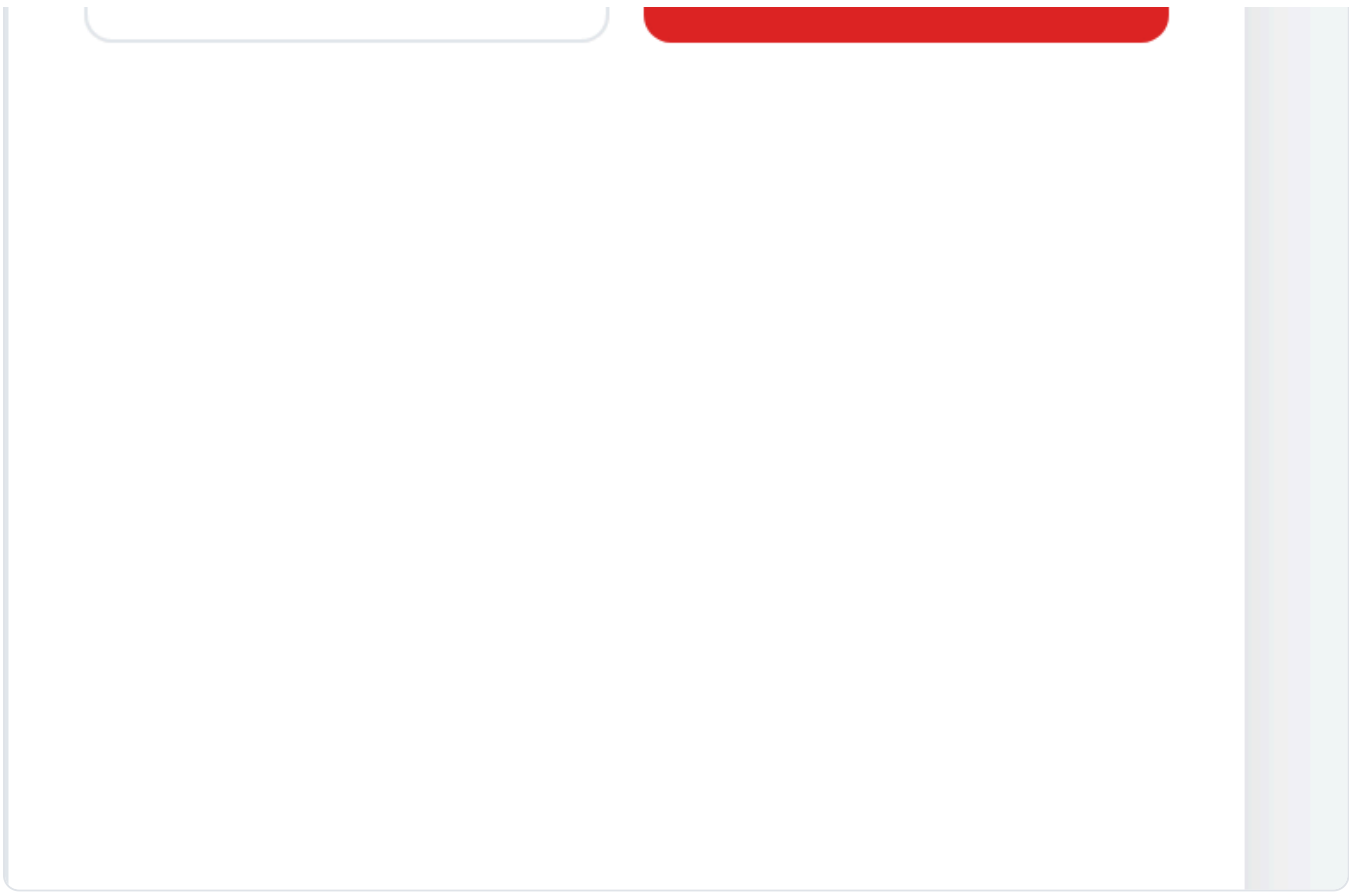


Figure 6 — Details (mobile): the details panel overlays the full screen.


NEUMANN
+ Add
MT
Mitsue Tollner
Job title
Software Developer
Company
Benton
City
Chicago
County
Orleans
Email
—
Bio
Generate with AI
No bio yet.
Edit
Delete



Appendix A — Seed data

These employees are inserted on first run. (`Photo*` marks records that have a sample image; others fall back to a generated avatar.) An **admin** user is also seeded — see the README for credentials.

First	Last	Company	Address	City	County	Color	Photo
James	Butt	Benton	6649 N Blue Gum St	New Orleans	Orleans	`#8bc447`	✓
Josephine	Darakjy	Chanay	4 B Blue Ridge Blvd	Brighton	Livingston	`#8a3b93`	
Art	Venere	Chemel	8 W Cerritos Ave #54	Bridgeport	Gloucester	`#1473bb`	
Lenna	Paprocki	Feltz Printing	639 Main St	Anchorage	Anchorage	`#c32482`	
Donette	Foller	Feltz Printing	34 Center St	Hamilton	Butler	`#c32482`	
Simona	Morasca	Chanay	3 Mcauley Dr	Ashland	Ashland	`#8a3b93`	
Mitsue	Tollner	Benton	7 Eads St	Chicago	Cook	`#8bc447`	
Leota	Dilliard	Commercial Press	7 W Jackson Blvd	San Jose	Santa Clara	`#dde553`	
Sage	Wieser	Feltz Printing	5 Boston Ave #88	Sioux Falls	Minnehaha	`#c32482`	✓
Kris	Marrier	Feltz Printing	228 Runamuck Pl #2808	Baltimore	Baltimore	`#c32482`	
Minna	Amigon	Chanay	2371 Jerrold Ave	Kulpsville	Montgomery	`#8a3b93`	✓
Abel	Maclead	Chemel	37275 St Rt 17m M	Middle Island	Suffolk	`#1473bb`	
Kiley	Caldarera	Chemel	25 E 75th St #69	Los Angeles	Los Angeles	`#1473bb`	
Bette	Ruta	Benton	98 Connecticut Ave Nw	Chagrin Falls	Geauga	`#8bc447`	
Veronika	Albares	Benton	56 E Morehead St	Laredo	Webb	`#8bc447`	

> The machine-readable version lives at `backend/app/seed_data.py` in the reference solution.

Appendix B — Curated learning links

Stack docs - React + TypeScript: https://react.dev/learn/typescript - Vite: https://vitejs.dev/guide/ - TanStack Query: https://tanstack.com/query/latest - React Router: https://reactrouter.com/ - FastAPI: https://fastapi.tiangolo.com/ - Pydantic v2: https://docs.pydantic.dev/latest/ - SQLAlchemy 2.0: https://docs.sqlalchemy.org/en/20/ - Alembic (migrations): https://alembic.sqlalchemy.org/ - PostgreSQL: https://www.postgresql.org/docs/ · MySQL: https://dev.mysql.com/doc/ - Docker Compose: https://docs.docker.com/compose/

Testing - pytest: https://docs.pytest.org/ · FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/ - Vitest: https://vitest.dev/ · React Testing Library: https://testing-library.com/docs/react-testing-library/intro/ - Playwright: https://playwright.dev/

Concepts - JWT: https://jwt.io/introduction · CORS: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS - REST APIs: https://developer.mozilla.org/en-US/docs/Glossary/REST - Writing test cases (example): https://www.guru99.com/test-case.html

AI-assisted development - Anthropic API: https://docs.claude.com/ · Claude Code: https://docs.claude.com/en/docs/claude-code

---